The Four Pillars of Object-Oriented Programming (OOP): Abstraction, Encapsulation, Inheritance, and Polymorphism.

I still remember the first time I came across the **Four Pillars of Object-Oriented Programming (OOP)**. I had always believed hard work guaranteed success, until that moment challenged everything I thought I knew, I would code and create a fully working program without knowing why am I using certain methods but because I needed them, I would work with those functions. Failure, I later learned, can be one of our greatest teachers, especially because at times the program would work and sometimes the code wouldn't work, researching these errors I later learned that I needed to know how to make use of the OOP.

Let's talk about the "The four principles of object-oriented programming", these are the features that if used properly, OOP can help you write more testable, flexible, and maintainable code. At my previous school OOP was self-taught and thru research that's where I discovered that OOP is designed into 3 parts, 1. State (what it knows), 2. Behaviour (What it can do), and 3. Identify (which instance you are working with). It is then divided into four principles which is abstraction, inheritance, encapsulation, and polymorphism.

Abstraction

Abstraction is a process of hiding implementation details and exposing only the functionality to the user for an example would be a TV remote, are you *really* aware of *how* your TV turns on when you press the ON button on the remote? Do you, as a user, need to know the specific sequence of 0's and 1's that your remote control emits to signal to the television's receiver that it should turn on? Or you just want to see the TV ON and playing what you want to watch? Yes, we only want what we want to watch, and this shows us that, in abstraction, we deal with ideas and not events. This means that the user will only know "what it does" rather than "how it does", only showing what is important.

Encapsulation

Encapsulation is "the action of enclosing something in or as if in a capsule", for example would be a pill which it keeps related things together and protects them from being messed with by external things. Removing access to parts of your code and making things private is exactly what encapsulation is all about (often times, people refer to it as data hiding), you control what gets in and out. Every object in encapsulation controls its own state, instead of letting outside code reach in and change its variables directly, the object decides if, when, and how those changes happen.

Inheritance

This one has always been my favourite, it is so nice to reuse code or extend it for more specific use cases, and that's what inheritance is about. Ever wished you could inherit from your parents, maybe their car, house or business, well in programming all this is possible, one class can "inherit" features from another which makes it easier instead of starting from stretch. We know sometimes that multiple places need to do the same thing, and they need to do everything the same except for one small part. This is a problem inheritance can solve. The best practice for this, is never inherit from a class that is unrelated to what you working on just because of one method you need, reason being it can get confusing.

Polymorphism

Polymorphism is the ability to perform many things in many ways. The word Polymorphism is from two different Greek words- poly and morphs. "Poly" means many, and "Morphs" means forms. So polymorphism means many forms, doing one thing in a lot of different ways, an example could be an ATM, you can withdraw, deposit get your statement, etc. The real power of polymorphism is sharing behaviours, and allowing custom overrides. A class with many methods and functions has to implement each as per the situation and the conditions.

Looking back, my journey with the Four Pillars of OOP started as a struggle to understand why my code would break for no apparent reason. I used to think that as long as the program ran, I had succeeded. However, I've learned that there is a massive difference between code that works and code that is well-built. By moving away from "working with functions" simply because I needed them and instead embracing State, Behaviour, and Identity, I've gained a much deeper control over my craft.

Abstraction and Encapsulation have taught me how to simplify complex systems and protect my data, while Inheritance and Polymorphism have shown me how to write elegant, reusable code. These aren't just technical terms but they are the foundation that keeps a programmer from feeling overwhelmed by their own creation. My experience taught me that while hard work is the engine, these four principles are the steering wheel, they don't just help you code more, they help you code better.

Bibliography Table

Source	Reference
GeeksforGeeks (Java OOP concepts)	GeeksforGeeks. (n.d.). Four main object-oriented programming concepts of Java. GeeksforGeeks. From https://www.geeksforgeeks.org/java/four-main-object-oriented-programming-concepts-of-java/
FreeCodeCamp (Four pillars of OOP)	FreeCodeCamp. (2020, September 14). The four pillars of object-oriented programming. FreeCodeCamp. From https://www.freecodecamp.org/news/four-pillars-of-object-oriented-programming/ (freecodecamp.org in Bing)
Khalil Stemmler (OOP principles)	Stemmler, K. (2019, October 21). The four principles of object-oriented programming. Khalil Stemmler. From https://khalilstemmler.com/articles/object-oriented/programming/4-principles/ (khalilstemmler.com in Bing)
GeeksforGeeks (Perl polymorphism)	GeeksforGeeks. (n.d.). Perl polymorphism in OOPs. GeeksforGeeks. From https://www.geeksforgeeks.org/perl/perl-polymorphism-in-oops/ +